Sistemas Operacionais

Sincronização de processos e threads
Parte 1: por que sincronizar?

Prof. Frederico Luís Cabral

Universidade Federal de Ciência e Inovação do Rio de Janeiro (UFCIRJ)

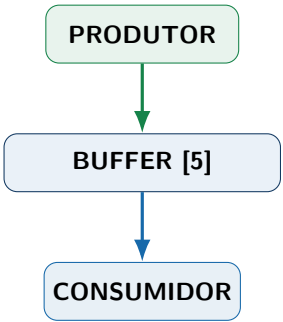
Bacharelado em Ciência da Computação

github.com/fredluiscabral/UFCIRJ/tree/main/SO

13 de outubro de 2026

Uma situação real: produzir e consumir dados

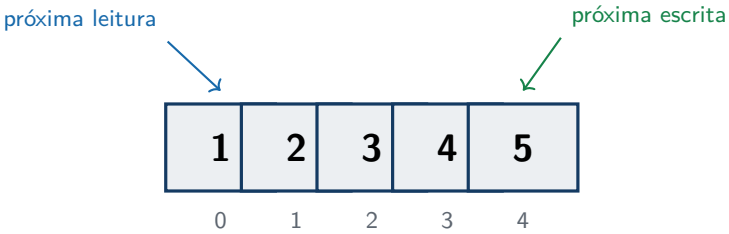
- Uma **thread produtora** gera itens.
- Uma **thread consumidora** processa itens.
- Ambas acessam um **buffer compartilhado**.
- O buffer tem **capacidade finita**: 5 posições.



Pergunta para a turma

O que pode acontecer quando as duas threads executam concorrentemente?

O buffer circular



- Índice de escrita: `entrada = (entrada + 1) % 5`
- Índice de leitura: `saida = (saida + 1) % 5`
- Ao chegar ao fim, cada índice **volta à posição zero**.

Questão

O índice circular, sozinho, impede a sobrescrita de dados não consumidos?

Primeira implementação: dados compartilhados

```c
#define TAM_BUFFER 5
#define TOTAL_ITENS 20

int buffer[TAM_BUFFER] = {0};
int entrada = 0;
int saida = 0;
```

- `buffer`: vetor visível às duas threads.
- `entrada`: posição onde o produtor escreverá.
- `saida`: posição de onde o consumidor lerá.
- Valor `0`: marcador didático de posição vazia (itens válidos: 1 a 20).

O código completo está no arquivo `produtor_consumidor.c` do projeto.

O produtor: escreve sem verificar espaço livre

```
void *produtor(void *arg) {
    for (int item = 1; item <= TOTAL_ITENS; ++item) {
        usleep(100000);           // 100 ms
        int pos = entrada;
        buffer[pos] = item;       // escreve SEM verificar!
        entrada = (entrada + 1) % TAM_BUFFER;
        printf("PRODUZ %d -> posicao %d\n", item, pos);
        mostrar_buffer();
    }
    return NULL;
}
```

Observe

O produtor pode escrever em uma posição que ainda guarda um item não consumido.

O consumidor: lê sem verificar se há item

```c
void *consumidor(void *arg) {
    for (int i = 0; i < TOTAL_ITENS; ++i) {
        usleep(150000);            // 150 ms
        int pos = saida;
        int item = buffer[pos];    // le SEM verificar!
        buffer[pos] = 0;           // limpa visualmente
        saida = (saida + 1) % TAM_BUFFER;
        printf("CONSOME %d <- posicao %d\n", item, pos);
        mostrar_buffer();
    }
    return NULL;
}
```

Observe

O consumidor pode ler uma posição vazia, ou receber um item diferente daquele esperado.

As duas threads executam concorrentemente

```
int main(void) {
    pthread_t tprod, tcons;

    pthread_create(&tprod, NULL, produtor, NULL);
    pthread_create(&tcons, NULL, consumidor, NULL);

    pthread_join(tprod, NULL);
    pthread_join(tcons, NULL);
    return 0;
}
```

- `pthread_create`: inicia cada thread.
- `pthread_join`: faz a thread principal aguardar o término.
- **Nenhuma dessas chamadas protege o buffer.**

Evolução possível do buffer (intercalação ilustrativa)

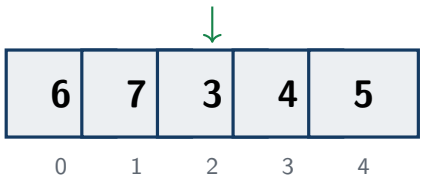
Operação	Buffer após a operação	Observação
Produz 1	`[ 1][ -][ -][ -][ -]`	escreve na posição 0
Consome 1	`[ -][ -][ -][ -][ -]`	retira da posição 0
Produz 2	`[ -][ 2][ -][ -][ -]`	
Produz 3	`[ -][ 2][ 3][ -][ -]`	
Consome 2	`[ -][ -][ 3][ -][ -]`	
Produz 4	`[ -][ -][ 3][ 4][ -]`	
Produz 5	`[ -][ -][ 3][ 4][ 5]`	
Produz 6	`[ 6][ -][ 3][ 4][ 5]`	

Esta sequência é ilustrativa: a ordem real varia conforme o escalonamento. Não é uma transcrição de uma execução observada.

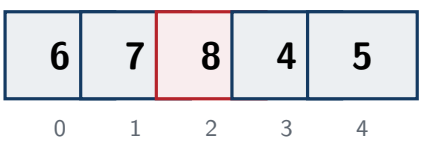
O momento da falha: um item é sobrescrito

O consumidor ainda não leu o item 3.

Produtor escreverá o item 8 na posição 2



O produtor escreve sem esperar por espaço livre:

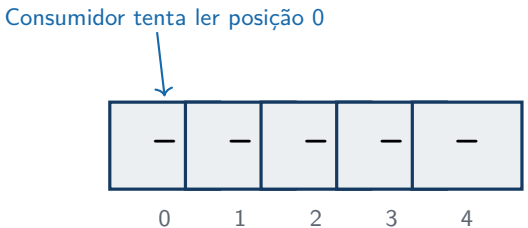


O que aconteceu com o item 3?

O item **8** ocupou a mesma posição: o item **3** foi perdido antes da leitura.

Segundo problema: e se o buffer estiver vazio?

Agora considere o consumidor mais rápido que o produtor.



Pergunta

O que o consumidor recebe se ele lê uma posição que ainda não contém um novo item?

Cenário ilustrativo: pode ocorrer na versão sem sincronização ao alterar as velocidades relativas, por exemplo, consumidor a 50 ms e produtor a 150 ms.

Buffer vazio: o consumidor lê sem esperar

Evento	Estado após o evento
CONSOME 0 ← pos. 0	[-][-][-][-][-]
CONSOME 0 ← pos. 1	[-][-][-][-][-]
PRODUZ 1 → pos. 0	[1][-][-][-][-]

```
int pos = saida;
int item = buffer[pos];   // pode ler 0: posicao vazia!
buffer[pos] = 0;
saida = (saida + 1) % TAM_BUFFER;
```

O defeito

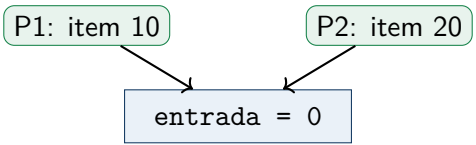
Não existe uma espera por **item disponível**. O consumidor pode avançar a posição de leitura sem ter consumido nenhum item válido.

Sequência possível, não captura de execução. Zero é marcador didático de vazio.

Terceiro problema: condição de corrida no índice

Uma pequena variação do sistema

Agora temos **dois produtores**, P1 e P2, compartilhando a variável global `entrada`. Ambos querem inserir um item.



O que muda?

Diferentemente do caso anterior, **duas threads modificam a mesma variável de posição**, sem exclusão mútua.

O trecho vulnerável: ambos usam `entrada`

```
// Executado tanto por P1 quanto por P2
int pos = entrada;                  // leitura compartilhada
buffer[pos] = item;                 // escrita no buffer
entrada = (entrada + 1) % TAM_BUFFER; // atualizacao
```

Por que isso não é uma única operação?

Uma thread pode ser interrompida entre a leitura de `entrada` e sua atualização. Nesse intervalo, outra thread pode ler o **mesmo valor**.

As duas podem escolher a mesma posição!

Intercalação: duas threads escolhem a posição 0

Passo	Produtor P1 (item 10)	Produtor P2 (item 20)
1	`pos = entrada` (lê 0)	
2		`pos = entrada` (lê 0)
3	`buffer[0] = 10`	
4		`buffer[0] = 20`
5	`entrada = 1`	
6		`entrada = 1`



Resultado desse entrelaçamento

O item **10 foi sobrescrito** por 20, e o índice avançou apenas até 1. Os dois produtores acreditavam ter reservado a mesma posição.

Três falhas, três perguntas

1. **Buffer cheio:** como impedir que o produtor sobrescreva itens pendentes?
2. **Buffer vazio:** como impedir que o consumidor leia sem haver item disponível?
3. **Condição de corrida:** como garantir que dois produtores não escolham a mesma posição?

Duas necessidades distintas

Coordenação (esperar por espaço ou itens) e **exclusão mútua** (proteger atualizações concorrentes).

Em C, acessos conflitantes não atômicos e sem sincronização caracterizam uma data race e geram comportamento indefinido.

Próximos conceitos

1. **Região crítica**: que trechos acessam dados compartilhados?
2. **Exclusão mútua**: como impedir interferências perigosas?
3. **Coordenação**: como aguardar *buffer não vazio* ou *buffer não cheio*?
4. **Primitivas**: mutex e semáforos.

Mensagem principal

Ter duas threads em execução não garante que elas cooperem corretamente. **É necessário sincronizar.**

Região Crítica

Definição

Uma **região crítica** é um trecho de código que acessa recursos compartilhados e cuja execução concorrente precisa ser controlada para preservar a correção do programa.

Exemplos de recursos compartilhados:

- Variáveis e estruturas de dados em memória;
- Buffers e filas compartilhadas;
- Arquivos e dispositivos de entrada e saída;
- Estruturas internas do Sistema Operacional.

Problema

A execução concorrente de operações sobre um recurso compartilhado pode produzir resultados incorretos quando não existe sincronização adequada.

Região Crítica: Produtor-Consumidor

Produtor

```
int posicao = entrada;

buffer[posicao] = item;

entrada = (entrada + 1)
          % TAM_BUFFER;
```

Consumidor

```
int posicao = saida;

int item = buffer[posicao];

buffer[posicao] = 0;

saida = (saida + 1)
        % TAM_BUFFER;
```

O que existe em comum?

Ambas as threads acessam o mesmo `buffer`.
Se acessarem simultaneamente a mesma posição, poderão ocorrer interferências entre suas operações.

Exclusão Mútua

Definição

A **exclusão mútua** é uma propriedade de sincronização que garante que, em um determinado instante, **no máximo uma thread ou processo** execute uma região crítica protegida pelo mesmo mecanismo.

Objetivo:

- Evitar acessos conflitantes simultâneos;
- Preservar a consistência dos dados compartilhados;
- Impedir interferências entre operações críticas.

Ideia fundamental

Se uma thread está executando a região crítica, outra thread que deseja executar a mesma região protegida deve aguardar.

Estrutura de uma Solução com Exclusão Mútua

Estrutura geral

```
while (1) {

    /* Regiao de entrada */
    /* REGIAO CRITICA */
    /* Regiao de saida */
    /* Regiao restante */

}
```

- **Região de entrada:** solicita permissão para acessar a região crítica.
- **Região crítica:** acessa ou modifica os recursos compartilhados.
- **Região de saída:** libera a região crítica.
- **Região restante:** executa operações que não precisam daquela exclusão mútua.

Exclusão Mútua é Suficiente?

Considere novamente o problema do produtor-consumidor.

Mesmo que produtor e consumidor nunca acessem simultaneamente o buffer:

- O produtor ainda pode tentar inserir um item quando o buffer estiver **cheio**.
- O consumidor ainda pode tentar retirar um item quando o buffer estiver **vazio**.

Conclusão

Exclusão mútua não é suficiente!
Além de controlar o acesso concorrente, precisamos **coordenar a execução** das threads de acordo com o estado do buffer.

Taxonomia das Soluções de Exclusão Mútua

Critério 1: Como a thread espera?

- **Espera ocupada (busy waiting):** repete testes usando CPU.
- **Espera com bloqueio:** suspende a execução até poder prosseguir.

Critério 2: Qual é o mecanismo empregado?

- **Algoritmos clássicos por software:** alternância estrita e Peterson.
- **Primitivas atômicas:** Test-and-Set, Compare-and-Swap.
- **Abstrações de biblioteca/SO:** mutexes e outras primitivas.

Atenção

Os dois critérios são **independentes**: um mutex pode usar operações atômicas e, dependendo da implementação, bloquear a thread.

Primeira Tentativa: Variável de Controle

Considere uma variável compartilhada:

Pseudocódigo

```
int ocupado = 0;
while (ocupado == 1);
ocupado = 1;
/* Regiao critica */
ocupado = 0;
```

- `ocupado = 0`: região crítica livre.
- `ocupado = 1`: região crítica ocupada.
- A thread espera enquanto a região estiver ocupada.

Pergunta

Essa solução realmente garante exclusão mútua?

Por Que a Primeira Tentativa Falha?

Suponha inicialmente:

$$ocupado = 0$$

Passo	Thread 1	Thread 2
1	Lê ocupado $= 0$	
2		Lê ocupado $= 0$
3	ocupado $= 1$	
4		ocupado $= 1$
5	Entra na região crítica	
6		Entra na região crítica

Falha

As duas threads entram na região crítica!
A verificação e a atualização da variável ocupado não formam uma operação atômica.

Espera Ocupada (Busy Waiting)

Definição

A thread verifica repetidamente uma condição para tentar entrar na região crítica, sem se bloquear voluntariamente.

Exemplo conceitual

```
while (ocupado)
    ;  /* continua testando */
```

- Consome tempo de CPU durante a espera.
- Aparece na primeira tentativa, na alternância estrita, em Peterson e nos spinlocks.
- **Não implica, por si só, incorreção:** Peterson é correto sob as hipóteses do modelo teórico.

Requisitos de uma Solução Correta

Uma solução clássica para o problema da região crítica deve satisfazer três propriedades.

1. Exclusão Mútua

No máximo uma thread pode executar a região crítica em determinado instante.

2. Progresso

Se nenhuma thread está na região crítica, a escolha de quem entrará não pode ser adiada indefinidamente pelas threads que não desejam entrar.

3. Espera Limitada

Após uma thread solicitar entrada, deve existir um limite para o número de entradas das outras threads antes dela.

Segunda Tentativa: Alternância Estrita

Considere duas threads identificadas por `0` e `1`.

Variável compartilhada

```
int vez = 0;
```

Thread 0

```
while (vez != 0);
/* Regiao critica */
vez = 1;
```

Thread 1

```
while (vez != 1);
/* Regiao critica */
vez = 0;
```

Ideia

As threads recebem permissão para entrar na região crítica alternadamente, com **espera ocupada**.

O Problema da Alternância Estrita

Suponha que:

$$vez = 0$$

1. A Thread 0 entra na região crítica.
2. Ao sair, executa `vez = 1`.
3. A Thread 1 não deseja entrar na região crítica.
4. A Thread 0 tenta entrar novamente.
5. A Thread 0 permanece esperando, pois `vez = 1`.

Problema

A Thread 0 fica impedida de entrar, mesmo com a região crítica livre.

A solução garante exclusão mútua, mas não satisfaz o requisito de progresso.

Algoritmo de Peterson

O algoritmo de Peterson utiliza duas informações:

- Se uma thread deseja entrar na região crítica;
- Qual thread recebe prioridade em caso de disputa.

Variáveis compartilhadas

```
bool interesse[2] = {false, false};

int vez = 0;
```

- `interesse[i]` indica que a thread `i` deseja entrar.
- `vez` é utilizada para resolver a disputa quando ambas desejam entrar.

Algoritmo de Peterson: Pseudocódigo

Para a thread `i`, com $j = 1 - i$:

Algoritmo teórico

```
interesse[i] = true;
vez = j;
while (interesse[j] && vez == j)
    ;
/* Regiao critica */
interesse[i] = false;
```

- A thread anuncia seu interesse e cede prioridade à outra.
- O laço `while` é uma forma de **espera ocupada**.
- Ao sair, a thread retira seu interesse.

Por Que Peterson Funciona?

Considere duas threads tentando entrar simultaneamente.

1. Ambas indicam interesse em entrar.
2. Ambas atualizam a variável vez.
3. A última atualização determina qual thread deverá ceder prioridade.
4. Uma thread entra na região crítica.
5. A outra aguarda até que a primeira retire seu interesse.

Resultado

No modelo clássico de memória sequencialmente consistente, Peterson satisfaz exclusão mútua, progresso e espera limitada para duas threads.

Das Soluções Clássicas ao OpenMP

O algoritmo de Peterson demonstra como garantir exclusão mútua utilizando variáveis compartilhadas.

Entretanto, sua utilização prática apresenta limitações:

- A versão estudada funciona com duas threads.
- Utiliza espera ocupada.
- Depende de garantias de atomicidade e ordenação.
- Exige implementar o protocolo de sincronização.

Pergunta

Precisamos implementar algoritmos como Peterson sempre que quisermos proteger uma região crítica?

Introdução ao OpenMP

Definição

OpenMP é uma API para programação paralela em memória compartilhada, utilizada com C, C++ e Fortran.

Características:

- Permite criar regiões de execução paralela.
- Utiliza threads que compartilham memória.
- Oferece diretivas para distribuir trabalho.
- Disponibiliza mecanismos de sincronização.

Diretivas

Em C, as diretivas OpenMP são introduzidas por `#pragma omp`.

OpenMP: Primeiro Programa

```
#include <stdio.h>
#include <omp.h>
int main(void)
{
    #pragma omp parallel num_threads(4)
    {
        int id = omp_get_thread_num();
        printf("Thread %d\n", id);
    }
    return 0;
}
```

Compilação e execução

```
gcc -fopenmp exemplo.c -o exemplo
./exemplo
```

- `parallel`: cria uma região paralela.

Como as Threads Executam?

Uma possível saída

```
Thread 0
Thread 2
Thread 1
Thread 3
```

- Todas as threads executam o bloco delimitado pela região `parallel`.
- A ordem das mensagens não é determinada.
- As threads podem executar simultaneamente em diferentes núcleos.

Importante

A existência de múltiplas threads não garante uma ordem específica de execução.

Variáveis Compartilhadas e Privadas

```
int compartilhada = 0;
#pragma omp parallel num_threads(4)
{
    int privada = omp_get_thread_num();
    #pragma omp critical
    {
        compartilhada++;
    }
}
```

Variável compartilhada

`compartilhada` é acessada pelas quatro threads.

Variável privada

Cada thread possui sua própria variável local `privada`.

Condição de Corrida com OpenMP

Considere o seguinte programa:

```
long contador = 0;
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 1000000; i++) {
    contador++;
}
printf("Contador = %ld\n", contador);
```

Resultado esperado:

$$contador = 1.000.000$$

Problema

As quatro threads modificam a mesma variável `contador` sem sincronização.
A operação `contador++` não é atômica.

OpenMP: Diretiva Critical

A diretiva `critical` permite proteger uma região crítica.

```
long contador = 0;
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 1000000; i++) {
    #pragma omp critical
    {
        contador++;
    }
}
```

Funcionamento

Apenas uma thread por vez pode executar a região `critical` correspondente.

Resultado

O incremento passa a ser protegido por exclusão mútua.

Como Funciona a Diretiva Critical?

Considere quatro threads tentando executar a mesma região crítica.

Thread	Situação
Thread 0	Executando a região crítica
Thread 1	Aguardando
Thread 2	Aguardando
Thread 3	Aguardando

Quando a Thread 0 termina, outra thread pode entrar na região crítica.

Propriedade garantida

No máximo uma thread executa a mesma região crítica protegida em determinado instante.

Observação

A diretiva não estabelece uma ordem específica para a entrada das threads.

OpenMP: Diretiva Atomic

Quando precisamos proteger uma atualização simples, podemos utilizar a diretiva `atomic`.

```
long contador = 0;
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 1000000; i++) {
    #pragma omp atomic
    contador++;
}
```

Funcionamento

A diretiva garante a atomicidade da atualização de `contador` em relação a acessos atômicos compatíveis.

- É aplicável a operações específicas.
- Pode ser mais eficiente que `critical`.
- Não protege arbitrariamente um bloco de código.

OpenMP: Diretiva Reduction

Outra possibilidade é evitar que todas as threads atualizem diretamente o mesmo contador.

```
long contador = 0;
#pragma omp parallel for reduction(+:contador)
for (int i = 0; i < 1000000; i++) {
    contador++;
}
```

Funcionamento

Cada thread trabalha com uma cópia privada da variável de redução.
Ao final, os resultados são combinados utilizando a operação especificada.

Diferença fundamental

A redução evita a disputa direta pelo contador durante as iterações, em vez de proteger cada incremento com exclusão mútua.

Comparando as Soluções em OpenMP

Solução	**Funcionamento**	**Aplicação**
`critical`	Exclusão mútua entre regiões correspondentes	Proteção de blocos de código
`atomic`	Atomicidade de operações específicas	Atualizações simples de variáveis
`reduction`	Cópias privadas e combinação dos resultados	Somas, contagens e outras reduções

Conclusão

Nem toda sincronização precisa ser implementada por meio de uma região crítica explícita.
A escolha do mecanismo depende da operação e dos requisitos do programa.

Da exclusão mútua à coordenação

Retomando o produtor-consumidor

Proteger o acesso ao buffer evita interferências, mas não basta:

- o produtor precisa aguardar quando não há espaço;
- o consumidor precisa aguardar quando não há itens.

Nova questão

Como uma thread pode **esperar por uma condição** sem consultar a memória continuamente?

Semáforo: conceito

Definição

Um **semáforo** é uma abstração de sincronização com um contador interno e operações atômicas de espera e sinalização.

- **wait(S)** (ou **P**/**down**): obtém uma permissão; caso não esteja disponível, aguarda.
- **signal(S)** (ou **V**/**up**): devolve uma permissão e pode despertar uma thread.

Observação

No modelo com bloqueio, aguardar não exige executar continuamente o teste; detalhes dependem da implementação.

Semáforo binário e semáforo contador

Binário

- Valores lógicos: 0 ou 1.
- Pode controlar acesso exclusivo.
- Não se confunde integralmente com um mutex: não possui necessariamente a noção de *proprietário*.

Contador

- Representa várias permissões.
- Ex.: cinco posições livres no buffer.
- Uma espera consome uma permissão; um sinal a repõe.

Exclusão mútua e coordenação usam a mesma abstração, com propósitos diferentes.

Operações de semáforo: pseudocódigo

Semáforo inicializado com $S = k$

`wait(S): obter uma permissão; se indisponível, aguardar`

`signal(S): devolver uma permissão; liberar eventual espera`

- **Atomicidade:** duas threads não podem consumir a mesma permissão.
- **Contagem:** o valor inicial representa recursos/permissões disponíveis.
- **Ordem:** o semáforo não impõe, em geral, FIFO ou ausência de inanição.

Importante

A implementação interna de `wait`/`signal` não é um simples `if` seguido de incremento/decremento comum.

Semáforos POSIX com threads OpenMP

Duas APIs com papéis diferentes

OpenMP cria e organiza as threads; **POSIX** fornece os semáforos `sem_t` para controlar o acesso aos recursos.

- `sem_init`: inicializa o semáforo.
- `sem_wait`: obtém permissão ou aguarda.
- `sem_post`: libera uma permissão.
- `sem_destroy`: destrói o semáforo.

Atenção

O semáforo é compartilhado pelas threads do mesmo processo. O parâmetro `pshared=0` indica esse uso entre threads.

OpenMP + semáforo POSIX: exclusão mútua

```
sem_t mutex;
long contador = 0;
sem_init(&mutex, 0, 1);
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 100000; ++i) {
    sem_wait(&mutex);
    contador++;
    sem_post(&mutex);
}
sem_destroy(&mutex);
```

Ponto central

Inicializar com 1 permite apenas uma thread por vez na região crítica. O código completo está em `exemplos/`.

Do semáforo binário ao contador

Semáforo binário: exclusão mútua

`mutex = 1`: restringe o número de threads que entram na região protegida.

Semáforos contadores: coordenação

No produtor-consumidor com buffer de tamanho N:

- `vazios = N`: número inicial de vagas.
- `cheios = 0`: número inicial de itens prontos.

Por que precisamos dos três?

`vazios` e `cheios` coordenam a produção e o consumo; `mutex` protege as operações na estrutura compartilhada.

OpenMP + POSIX: produtor-consumidor

Produtor

```
produzir(item);
sem_wait(&vazios);
sem_wait(&mutex);
inserir(item);
sem_post(&mutex);
sem_post(&cheios);
```

Consumidor

```
sem_wait(&cheios);
sem_wait(&mutex);
item = retirar();
sem_post(&mutex);
sem_post(&vazios);
consumir(item);
```

Execução

`#pragma omp parallel sections` cria as duas tarefas concorrentes. Código integral disponível no projeto.

Experimentos com OpenMP + POSIX

1. Execute a contagem com quatro threads e semáforo binário inicializado com 1.
2. Compare com uma versão sem proteção da atualização.
3. Execute o produtor-consumidor com um buffer de cinco posições.
4. Inverta as velocidades relativas do produtor e consumidor.

Questão

Por que a correção do contador precisa de exclusão mútua, enquanto o buffer limitado também precisa de coordenação?

Quatro problemas clássicos de sincronização

1. **Produtor-consumidor**: controlar espaços e itens em um buffer limitado.
2. **Leitores-escritores**: permitir leituras concorrentes sem conflito com escrita.
3. **Filósofos jantando**: evitar impasse na aquisição de múltiplos recursos.
4. **Barbeiro dorminhoco**: organizar clientes, cadeiras e atendimento.

Pergunta para todos os casos

Quais são os recursos, quais condições precisam valer e qual é o risco de sincronização incorreta?

Produtor-consumidor: invariantes do buffer

Buffer de capacidade N

- $0 \leq itens \leq N$.
- Produzir requer uma **posição livre**.
- Consumir requer um **item disponível**.
- Alterações nos índices e nas posições precisam preservar a consistência.

3 ocupadas; 2 livres



Produtor-consumidor: solução com semáforos

Inicialização: `vazios = N`, `cheios = 0`, `mutex = 1`.

Produtor

```
produzir(item);
wait(vazios);
wait(mutex);
inserir(item);
signal(mutex);
signal(cheios);
```

Consumidor

```
wait(cheios);
wait(mutex);
item = retirar();
signal(mutex);
signal(vazios);
consumir(item);
```

Por que esta ordem?

Aguardar por espaço ou item **antes** de adquirir o mutex evita segurá-lo durante uma espera por condição.

Leitores e escritores: o problema

Regras para um recurso compartilhado

- Vários leitores podem acessar **simultaneamente**.
- Um escritor precisa de acesso **exclusivo**.
- Leitor e escritor não podem acessar ao mesmo tempo quando há escrita.

$$L + L + L \checkmark \qquad L + E \times \qquad E + E \times$$

Questão de justiça

Se leitores chegarem continuamente, um escritor pode nunca conseguir acesso?

Leitores-escritores: semáforos (preferência aos leitores)

Inicialização: `mutex = 1`, `recurso = 1`, `nLeitores = 0`.

Leitor

```
wait(mutex);
nLeitores++;
if (nLeitores == 1) wait(recurso);
signal(mutex);
ler();
wait(mutex);
nLeitores-;
if (nLeitores == 0) signal(recurso);
signal(mutex);
```

Escritor

```
wait(recurso);
escrever();
signal(recurso);
```

Limitação

Pode haver **inanição de escritores**.

Filósofos jantando: o problema

Cenário

Cinco filósofos alternam entre pensar e comer. Cada um precisa adquirir **dois garfos vizinhos** para comer.

- Cada garfo só pode estar com um filósofo por vez.
- Se todos pegarem o garfo à esquerda e aguardarem o da direita, ninguém progride.

Conceito central

Deadlock (impasse): ciclo de espera por recursos que impede o progresso dos participantes.

Filósofos: solução conceitual com semáforos

Cinco semáforos binários `garfo[0..4] = 1` e um semáforo contador `sala = 4`.

Filósofo *i* (pseudocódigo)

```
pensar();
wait(sala);
wait(garfo[i]);
wait(garfo[(i+1) % 5]);
comer();
signal(garfo[(i+1) % 5]);
signal(garfo[i]);
signal(sala);
```

Garantia e limite

Restringir a quatro filósofos disputando garfos **evita o deadlock circular clássico**; não garante, por si só, ausência de inanição.

Barbeiro dorminhoco: o problema

Barbearia

Há um barbeiro, uma cadeira de atendimento e N cadeiras na sala de espera.

- Sem clientes, o barbeiro dorme.
- Ao chegar, um cliente acorda o barbeiro ou espera, se houver cadeira.
- Se todas as cadeiras estiverem ocupadas, o cliente vai embora.

Desafio

Evitar perda de avisos de chegada, excesso de clientes na sala e conflitos na contagem de cadeiras.

Barbeiro: solução conceitual com semáforos

Inicialização: `clientes = 0`, `barbeiro = 0`, `mutex = 1`, `esperando = 0`.

Cliente

```
wait(mutex);
if (esperando == N) {
  signal(mutex); sair();
} else {
  esperando++;
  signal(clientes);
  signal(mutex);
  wait(barbeiro);
  receber_corte();
}
```

Barbeiro

```
while (true) {
  wait(clientes);
  wait(mutex);
  esperando-;
  signal(barbeiro);
  signal(mutex);
  cortar_cabelo();
}
```

Protocolo didático: o sinal libera um cliente para atendimento; conclusão e saída podem exigir um aperto de mãos adicional.

Comparando os problemas clássicos

Problema	Condição de avanço	Risco central
Produtor-consumidor	Espaço para produzir; item para consumir	Sobrescrita / leitura inválida
Leitores-escritores	Escrita exclusiva; leitura compartilhada	Inanição por política de prioridade
Filósofos	Dois recursos vizinhos disponíveis	Deadlock e inanição
Barbeiro	Cliente presente; cadeira de espera livre	Perda de coordenação / capacidade

Síntese

Sincronização combina **segurança** (não violar invariantes) e **vivacidade** (permitir progresso).

Para discutir em sala

1. Por que o mutex sozinho não resolve o produtor-consumidor?
2. Por que a preferência a leitores pode prejudicar escritores?
3. Evitar deadlock garante que ninguém espere indefinidamente?
4. No barbeiro, o que distingue **ser chamado** de **terminar o corte**?

Próxima etapa

Implementar e testar protocolos de sincronização, investigando também políticas de justiça e desempenho.

Exemplos executáveis: quatro problemas clássicos

Programa	Semáforos e propósito
Produtor–consumidor	`vazios`, `cheios`, `mutex`: recursos e exclusão
Leitores–escritores	`recurso`, `mutex_contagem`: leitores simultâneos
Filósofos	`garfos[5]`, `sala`: evitar espera circular
Barbeiro	`chegadas`, `mutex`, sinais individuais

Como os programas foram organizados?

OpenMP cria as threads; semáforos POSIX controlam quem pode prosseguir.

Leitores e escritores: como compartilhar a leitura?

Entrada de um leitor

```
sem_wait(&mutex_contagem);
leitores_ativos++;
if (leitores_ativos == 1) sem_wait(&recurso);
sem_post(&mutex_contagem);
```

Escritor

```
sem_wait(&recurso);
dado = novo_valor;
sem_post(&recurso);
```

Pergunta

Por que somente o primeiro leitor bloqueia o escritor? Uma sequência contínua de leitores pode causar inanição do escritor.

Filósofos: por que limitar a sala?

Filósofo *i*

```
sem_wait(&sala);
sem_wait(&garfos[i]);
sem_wait(&garfos[(i+1)%5]);
comer();
sem_post(&garfos[(i+1)%5]);
sem_post(&garfos[i]);
sem_post(&sala);
```

Inicialização

Cinco garfos com valor 1; semáforo `sala` com valor 4.

Questão

Por que permitir no máximo quatro filósofos impede o deadlock circular clássico, mas não garante ausência de inanição?

Barbeiro dorminhoco: eventos e capacidade

- `mutex = 1`: protege a fila e o número de cadeiras ocupadas.
- `chegadas = 0`: conta clientes aguardando chamada.
- `chamado[id] = 0`: acorda o cliente correto.
- `concluido[id] = 0`: sinaliza que o corte terminou.

Fluxo

Cliente entra na fila $\rightarrow$ sinaliza chegada $\rightarrow$ espera chamada $\rightarrow$ espera conclusão.

Questão

Por que um único sinal de "barbeiro disponível" pode não identificar qual cliente será atendido?

Onde aparecem os problemas clássicos?

Os mesmos padrões aparecem em sistemas reais

- **Produtor–consumidor:** filas de impressão, streaming e processamento de requisições.
- **Leitores–escritores:** caches compartilhados, bancos de dados e sistemas de arquivos.
- **Filósofos jantando:** transações e threads que precisam adquirir vários recursos.
- **Barbeiro dorminhoco:** servidores com fila limitada e serviços que aguardam solicitações.

Ideia

Os personagens mudam, mas os desafios continuam: concorrência, coordenação e acesso seguro a recursos.

Produtor–consumidor em sistemas reais

Exemplo 1: fila de impressão

Aplicações enviam documentos (produtores); um serviço de impressão retira e processa trabalhos (consumidor).

Exemplo 2: streaming de vídeo

Uma thread recebe dados da rede; outra os decodifica. Um buffer absorve diferenças momentâneas de velocidade.

Exemplo 3: servidor com workers

Uma thread recebe requisições e outras retiram tarefas de uma fila limitada.

Pergunta

Se a fila estiver cheia, o que acontece com a thread produtora na nossa solução?

Leitores–escritores em sistemas reais

Exemplo 1: cache compartilhado

Muitas threads consultam dados em paralelo; atualizações exigem coordenação com as leituras.

Exemplo 2: banco de dados

Consultas e alterações concorrentes precisam preservar a consistência dos dados.

Exemplo 3: metadados de arquivos

Várias threads consultam informações; modificações precisam ser sincronizadas.

Atenção

Bancos de dados e sistemas de arquivos reais podem usar técnicas mais sofisticadas, como MVCC e bloqueios por granularidade.

Filósofos jantando em sistemas reais

Exemplo 1: transferência entre contas

Uma transação bloqueia a conta A e espera B; outra bloqueia B e espera A. Pode ocorrer **deadlock**.

Exemplo 2: aquisição de múltiplos mutexes

Thread 1 adquire `mutex1` antes de `mutex2`; thread 2 faz o contrário.

Estratégia prática

Estabelecer uma **ordem global de aquisição** dos recursos evita a formação de espera circular.

Pergunta

Por que todas as threads adquirirem os mutexes na mesma ordem ajuda a evitar deadlock?

Barbeiro dorminhoco em sistemas reais

Exemplo 1: servidor com fila limitada

Uma thread atende requisições; outras aguardam em fila. Se a fila estiver cheia, novas solicitações podem ser rejeitadas.

Exemplo 2: processamento de tarefas

Um worker aguarda trabalho sem consumir CPU continuamente. Uma chegada sinaliza que existe tarefa disponível.

Mapeamento da analogia

Barbeiro = worker; clientes = solicitações; cadeiras = capacidade da fila; corte = processamento.

Pergunta

Quando a fila está cheia, o sistema deve bloquear a chegada ou rejeitar o pedido?

Produtor–consumidor vs. barbeiro dorminhoco

Aspecto	**Produtor–consumidor**	**Barbeiro**
Fila/buffer limitado	Sim	Sim
Quando não há trabalho	Consumidor aguarda	Barbeiro dorme
Quando a fila enche	Produtor aguarda	Cliente pode ir embora
Principal ênfase	Disponibilidade de itens e vagas	Eventos de atendimento e capacidade

Discussão

Bloquear a solicitação e rejeitá-la são duas políticas possíveis de controle de sobrecarga.

Síntese: um mesmo servidor pode reunir os quatro

1. **Produtor–consumidor:** requisições entram em uma fila e são processadas por workers.
2. **Leitores–escritores:** requisições consultam ou atualizam um cache compartilhado.
3. **Filósofos:** algumas operações exigem vários locks e podem gerar deadlock.
4. **Barbeiro:** ao atingir a capacidade da fila, o servidor pode rejeitar novas solicitações.

Questão para a turma

Quais desses padrões poderiam ocorrer simultaneamente em um único servidor web?

Atividade de observação dos programas

1. Compile os cinco programas com `make` na pasta `exemplos`.
2. Execute cada programa pelo menos três vezes.
3. Compare a ordem das mensagens e o estado dos recursos.
4. Identifique o semáforo que protege dados e o que coordena eventos.
5. Discuta deadlock, inanição e limitações de justiça de cada política.

Discussão

O resultado correto depende da sincronização; a ordem exata de execução pode variar.